Your Server Has No Public IP. FedRAMP Still Calls It Reachable.

Matthew Venne
Chief Technology Officer, stackArmor

July 2026

A plain-language companion to "An Exclusion-Gated Method for Determining Internet Reachability of Vulnerabilities under the FedRAMP VDR/VER Rules."

The problem in one sentence

FedRAMP's new rules make one question — *is this vulnerability reachable from the internet?* — a major driver of how fast you must fix it, and then define "reachable" far more broadly than "the box has a public IP" — without telling you how to answer it.

The stakes are concrete. The remediation matrix gives internet-reachable findings a dramatically shorter clock than everything else; at the top of the matrix the difference is hours versus days, and further down it is days versus a month. Get the answer wrong in one direction and you burn your team on false emergencies. Get it wrong in the other and you have under-reported a federal finding.

So every provider needs a way to answer the question that is reproducible, fast enough to run on every scan, and strong enough to survive a skeptical 3PAO. That's what the paper builds.

What FedRAMP actually means by "reachable"

Two details in the rule text break most people's intuition.

First, **reachability follows the data, not the network diagram.** The rule explicitly includes systems that "have no direct route to/from the internet but receive payloads or otherwise take action triggered by internet activity." Log4Shell made this vivid: the classic victim was a thumbnail generator or log indexer on a private subnet, with no public IP and often no listening port at all. It just parsed files that internet users had uploaded. The malicious string rode in on the data, and the private box executed it. Under FedRAMP's definition, that box was internet-reachable the whole time.

Second, **reachability is a property of the vulnerability, not the server.** The rule says it "applies only to the specific vulnerable machine-based information resources processing the payload." The same host can carry one flaw that is internet-reachable and another that is not, because the two flaws sit in different code, listen on different surfaces, and are triggered by different things. Answering per-host is answering the wrong question.

Put those together and the naive shortcuts collapse. "It's on a private subnet" answers neither point. What you need is a method that traces where internet data actually goes, and then asks, for each individual flaw, whether that data can set it off.

The rule of the method: reachable until proven otherwise

The method's central design choice is the direction of proof. **You never prove a vulnerability is reachable. Every vulnerability starts out internet-reachable, and it leaves that list only by passing a hard-evidence check.** Missing telemetry, an unknown protocol, an unclassifiable bug — every evidence gap defaults to "still reachable." Gaps can only make the urgent list longer, never quietly shorter.

That sounds harsh, but it is the property that makes the whole thing defensible: a false "not reachable" hides a federal finding on a slow clock, while a false "reachable" merely wastes some urgency. The method is built so that every exclusion is a deliberate, evidenced decision — and everything else stays hot.

The checks are organized as a pipeline of three phases:

THE WHOLE PIPELINE

Phase A — direct exposure. Five gates decide which assets the internet can talk to at all → **Phase B — payload exposure.** Follow the data from those entry points through queues, uploads, and replication; everything the data touches stays in scope → **Phase C — per-flaw filters.** For each vulnerability on an exposed asset, three questions decide whether internet data can actually trigger *this* flaw → the verdict, per vulnerability: internet-reachable (IRV) or not (NIRV) — with every check defaulting to "reachable" when evidence is missing.

Phase A: five gates on the front door

Phase A asks which assets an outside attacker can exchange data with directly. It is five gates, each a question with a deterministic answer:

1. **Does the internet route to it at all?** Public IPs, load balancers, ingress paths.
2. **Does the firewall actually stop strangers?** The test is *attribution*, not size: an allow-rule is only an exclusion if every source range in it can be named and claimed — your offices, your VPN egress, your monitoring vendor. A rule that happens to be narrow but points at address space nobody can identify excludes nothing.
3. **Whose login is in front of it?** This is where intuitions most often fail. A VPN or identity-aware proxy in front of an *enrolled workforce* — issued credentials, managed devices, named principals — is a real boundary, and it excludes. But when the login wall is the front door of the customer application, open to the application's general internet user base, authentication excludes nothing: internet data still flows through the login and into the backend. SQL injection behind a customer sign-in page is still internet-reachable. (What the login *does* earn you is an exploitability argument — it defeats FedRAMP's "an automated unauthenticated system can exploit it" floor — which shortens nothing about the data path but can support a likely-exploitable downgrade.)
4. **What does Kubernetes expose?** Ingress rules, gateway routes, and load-balancer services are readable directly from the cluster. The gate also covers what route-centric inventories miss — NodePorts that open a port on every node, host-network pods — and it refuses to guess: where the cluster alone cannot decide (a NodePort's fate depends on cloud firewall rules the cluster cannot see), the asset is treated as exposed until node-level evidence says otherwise.

5. **Does the vulnerable software actually hold the exposed port?** Not every package on an exposed host is exposed. If the flagged software maps, deterministically, to processes that hold no internet-facing socket, it is not *directly* exposed. But this gate has a hard limit — the Log4Shell shape from earlier. "Binds no exposed port" is never a standalone verdict; it only excludes together with Phase B's "and no internet data reaches it."

Phase B: follow the data, not the diagram

Everything Phase A marks as an entry point becomes a source of *taint*. Wherever internet-originated data flows — requests, queued messages, uploaded files, replication streams — every system that receives and processes it stays in scope, hop after hop. The private thumbnail worker is caught here: it is not an entry point, but it consumes what the entry points ingest.

The evidence is the union of two graphs: the flows your network policies and firewall rules *permit*, and the flows your telemetry actually *sees*. The two do different jobs. Observed flows can only ever add edges — seeing traffic proves a path exists, but not seeing traffic never proves one doesn't, because nothing stops a new flow from starting tomorrow. Only enforcement can rule paths out.

That asymmetry sets the bar for the method's one wholesale exclusion. If no path can reach an asset from any entry point, everything on it is not internet-reachable — but that claim must rest on deny-by-default rules that cover the asset, or on the operator's signed statement that the flow map is complete and no path leads there. "We watched the logs for a month and saw nothing" is not enough on its own: a monthly batch job, a failover path, or a disaster-recovery runbook can produce its first-ever flow the day after the window closes. Quiet logs prove the past, not the future.

Phase C: three questions about the flaw itself

An asset that internet data reaches is not an asset where every flaw matters. Phase C restores precision, one vulnerability at a time — honoring the rule's "only the specific vulnerable resources processing the payload." Three filters, each with a defined evidence source, each defaulting to "reachable" when the evidence is missing:

1. **Does the vulnerable code ever run?** Scanners flag what is *installed*; runtime telemetry shows what *executes*. Build leftovers, disabled agents, a vulnerable function in a library the application never calls — if execution evidence shows the code cannot run, internet data cannot trigger it.

2. **Does the arriving data even speak the flaw's language?** EternalBlue is an SMB file-sharing exploit. On a server whose only internet-facing surface is HTTPS, internet data arrives as HTTPS and is parsed as HTTPS — it never reaches SMB parsing code. A flaw is only reachable through data that arrives on the surface the flaw lives in. The catch that keeps this honest: for the many CVEs whose trigger surface can't be determined, the filter assumes the flaw is triggerable by *anything* — and the finding stays reachable.

3. **Does the bug travel in content, or does it need the attacker on the wire?** Most dangerous bugs are *content-triggered*: the exploit is the data itself — a SQL string, a poisoned image, a malicious log line — and it stays dangerous no matter how many internal hops it takes to arrive. Log4Shell, again. But some bugs are *peer-bound*: they live in the machinery

of the connection itself — a TLS handshake flaw, a low-level packet-parsing bug — and can only be triggered by the machine on the other end of the connection. If the vulnerable service's peers are all fixed internal systems (only the application server ever connects to the database), an internet attacker can send malicious *content* through the front door all day, but can never *become the database's connection peer* — and a handshake bug on that database is out of the attacker's reach. Content-triggered flaws get no such exclusion, ever, and any bug that can't be confidently classified is treated as content-triggered.

Every exclusion the pipeline grants maps to a standard, signable VEX justification — so each "not reachable" verdict ships with machine-readable evidence a 3PAO can check, not a spreadsheet note.

The common excuses, against the evidence the method actually demands:

"It's not reachable because..."	**What has to be true before that counts**
"it's behind the firewall"	every allowed source range is attributed to you or your customer — narrow-but-unclaimed rules don't count
"it's behind the VPN / SSO"	the population behind the login is your enrolled workforce, not the application's internet user base
"it has no public IP"	no data path reaches it either — proven by deny-by-default rules or an attested-complete flow map, not by quiet logs
"that package isn't even used"	runtime execution evidence shows the code never runs — installed-but-idle is a claim, telemetry is proof
"wrong protocol"	the flaw's trigger surface is known and provably absent from every surface internet data arrives on
"an attacker can't connect to it directly"	the flaw is peer-bound (connection machinery, not content) *and* the peers are fixed internal systems

The same flaw, two very different verdicts

The per-vulnerability grain is the point, so here it is in action:

- **SQL injection behind a customer login:** the login is the application's own front door, the malicious string is content, and it flows through authentication into the backend. **Internet-reachable** — even though the database has no public IP and sits three hops inside.
- **An SSH flaw on a public web server:** the same host is genuinely internet-facing — but port 22 is locked to the ops team's attributed addresses, the flaw lives in a surface internet data never arrives on, and the vulnerable process holds no public socket. **Not internet-reachable** — on the most public box in the estate.

Same network, opposite verdicts, and neither one is decided by where the server sits. The first is the false negative that per-host thinking produces; the second is the false positive that burns response teams for no security gain.

"But we have a WAF"

The question every provider asks next: our web application firewall blocks the exploit — surely the finding isn't internet-reachable anymore? FedRAMP does permit that reclassification when an interception demonstrably blocks the exploitation path. The paper's answer: you may, but you shouldn't — attest instead.

Log4Shell showed why in real time. Within days of the first WAF rules blocking the attack string, attackers published obfuscated variants that sailed straight past the patterns, and every WAF vendor spent December 2021 chasing them. A WAF exclusion is a bet that a signature matches every variant an adversary can invent; a topology exclusion is a bet that an attacker cannot send data over a connection that does not exist. Nobody obfuscates their way through a missing path. And the WAF bet also loses quietly to your own operations — a rule flipped to detection-only during an incident, a CDN migration — each silently voids the downgrade with nothing in the record prompting a second look.

So the recommended posture is: keep the finding internet-reachable, and ship a signed VEX statement naming the protecting control. If the WAF ever fails, the attested record is still true — internet-reachable, mitigation in doubt — while a reclassified record became false the moment the control failed, and nothing says so. The attestation is also simply better audit evidence: a signed, machine-readable assertion naming the exact control, instead of a quiet absence from the urgent list.

Two boundaries no mitigation moves. Known Exploited Vulnerabilities keep their CISA due dates "even if the vulnerability has been fully mitigated" — no virtual patch extends a KEV clock, and neither does a reclassification. And endpoint protection changes nothing about reachability in either mode: a detect-only EDR alerts after the payload has already arrived, and even a blocking-mode EDR that genuinely stops the exploit at runtime is evidence about *exploitability and impact* — worth real credit on those levers — not about whether internet data reaches the code. Geography and mitigation are different questions, and the method keeps them apart.

The bottom line

Four claims carry the whole paper:

1. **Reachability is about data paths, not IP addresses — and it attaches to the vulnerability, not the server.** A private box parsing uploaded files is reachable; a locked-down daemon on a public host may not be.

2. **You never prove "reachable."** Everything starts internet-reachable and leaves only past a hard-evidence check; every evidence gap defaults to urgent. False negatives hide federal findings; the method makes them expensive.

3. **Three phases, each deterministic:** five direct-exposure gates, taint-following through the data flows (pruned only by enforcement or attestation, never by quiet logs), and three per-flaw filters — with every exclusion carrying named evidence and a signable VEX justification.

4. **Mitigations don't change geography.** A WAF earns a signed attestation, not a reclassification; blocking EDR earns exploitability credit; KEV deadlines bend for nothing.

The full paper has the formal criteria, the Kubernetes and non-Kubernetes evidence sources, the worked examples, and the VEX mappings. But the idea you came for is simple: **stop asking**

"does this server face the internet?" and start asking "can internet data set off this flaw?" — and make every "no" show its evidence.

Read the full technical paper: An Exclusion-Gated Method for Determining Internet Reachability of Vulnerabilities under the FedRAMP VDR/VER Rules